

SOLID Principles in Object-Oriented Programming (OOP)

SOLID is a set of five design principles that help developers write clean, maintainable, and scalable object-oriented code. These principles improve software structure and make it easier to modify and extend applications without breaking existing functionality.

The five SOLID principles are:

- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

1. Single Responsibility Principle (SRP)

The Single Responsibility Principle states that a class should have only one responsibility and one reason to change. This makes the code easier to understand and maintain.

Example:

```
class ReportGenerator {  
    generate() {  
        console.log("Generating report...");  
    }  
}
```

```
class ReportPrinter {  
    print() {  
        console.log("Printing report...");  
    }  
}
```

```
class ReportSaver {  
    save() {  
        console.log("Saving report...");  
    }  
}
```

Each class performs a single task, which follows the SRP.

2. Open/Closed Principle (OCP)

The Open/Closed Principle states that software entities should be open for extension but closed for modification. This means we should be able to add new functionality without changing existing code.

Example:

```
class Discount {  
    calculate() {}  
}
```

```
class StudentDiscount extends Discount {  
    calculate() {  
        return 10;  
    }  
}
```

```
class RegularDiscount extends Discount {  
    calculate() {  
        return 5;  
    }  
}
```

New discount types can be added without modifying the original class.

3. Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that objects of a subclass should be replaceable with objects of the superclass without affecting the correctness of the program.

Example:

```
class Bird {}
```

```
class FlyingBird extends Bird {
```

```
    fly() {
```

```
        console.log("Flying...");
```

```
    }
```

```
}
```

```
class Sparrow extends FlyingBird {}
```

```
class Penguin extends Bird {}
```

Penguin does not inherit the flying behavior, so the substitution remains valid.

4. Interface Segregation Principle (ISP)

The Interface Segregation Principle states that clients should not be forced to depend on methods they do not use. Instead of one large interface, multiple smaller interfaces should be created.

Example:

```
class Workable {  
    work() {}  
}
```

```
class Eatable {  
    eat() {}  
}
```

```
class Human extends Workable {  
    work() {  
        console.log("Working");  
    }  
  
    eat() {  
        console.log("Eating");  
    }  
}
```

```
class Robot extends Workable {  
    work() {  
        console.log("Robot working");  
    }  
}
```

Each class implements only the functionality it needs.

5. Dependency Inversion Principle (DIP)

The Dependency Inversion Principle states that high-level modules should not depend on low-level modules. Both should depend on abstractions.

Example:

```
class Keyboard {  
    type() {  
        console.log("Typing...");  
    }  
}  
  
class Computer {  
    constructor(inputDevice) {  
        this.inputDevice = inputDevice;  
    }  
  
    start() {  
        this.inputDevice.type();  
    }  
}  
  
const keyboard = new Keyboard();  
const pc = new Computer(keyboard);  
pc.start();
```

The Computer class depends on an abstraction rather than a specific implementation.